

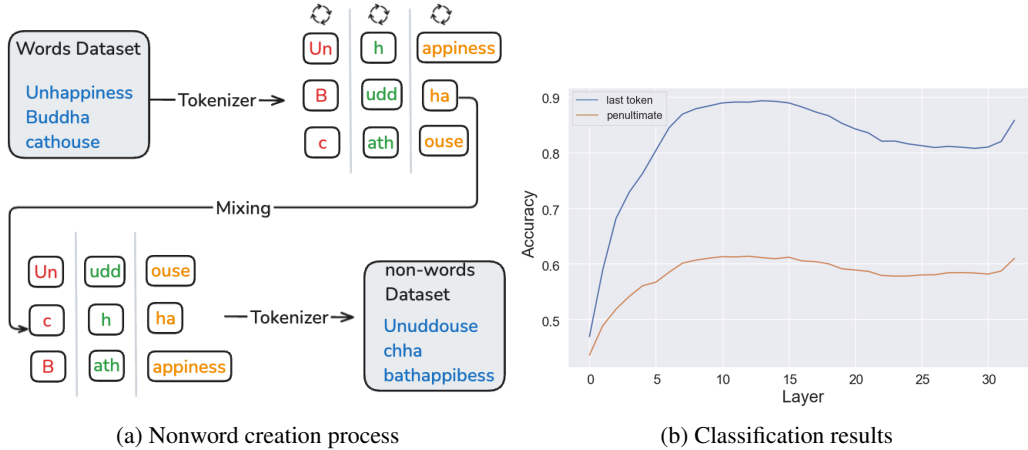


Figure 2: Our word vs. nonword probing classification experiments. (2a) **Dataset creation process.** Top: words from the Gutenberg corpus are tokenized using the Llama2 tokenizer. Bottom: nonwords are generated by shuffling tokens while maintaining their positions within the word; (2b) **Classification results of words vs. nonwords.** Using the **last** token shows strong results (reaching up to 90% accuracy), which sharply rise in early layers (3–7), peak in the middle (13), and decrease in later layers (20–32). Using the **penultimate** token shows substantially lower scores, suggesting that the high classification accuracy is specifically tied to the presence of a complete word rather than token co-occurrence patterns.

To address this question, we construct a balanced dataset containing two groups: one with real English words, and another with artificially generated, meaningless *nonwords* (Frisch et al., 2000). Both groups are tokenized using the Llama2 tokenizer (Touvron et al., 2023). The word dataset consists of 10,000 distinct words sampled from the Gutenberg corpus (Gerlach & Font-Clos, 2018), with 53% of the words containing two tokens, 37.3% containing three tokens, and the rest four tokens. We generate the nonwords by shuffling tokens from the word dataset, ensuring that the prefix and suffix positions align with the original tokens’ positional probabilities. For example, the token “ing”, extracted from the final position of real words in the dataset (e.g., “directing” tokenized as “direct”, “ing”), is retained as a suffix in the nonwords dataset. This process ensures that nonword tokens maintain position distribution properties similar to word tokens, preserving natural positional patterns and mitigating potential distributional biases (Fig. 2a). We next apply a k -nearest neighbors (k -NN) probing classifier ($k = 4$, using Euclidean distance) on the hidden states of the last tokens of both words and nonwords, for each layer of the Llama2-7B model. The training set consists of 80% of the dataset, and the remaining 20% are used for evaluation.

Our results (Fig. 2b, **blue**) reveal a three-stage pattern in the model’s representation of word and nonword token sequences. In the model’s first few layers, representations from both groups are relatively indistinguishable and accuracy is close to chance level. Then, from layers 2 to 6, a clear distinction between the two groups emerges, until the representations are almost completely separate in middle layers, between layers 6 and 20. At this point, the probe achieves a stable, high accuracy, peaking at 89% on layer 13. Finally, accuracy slightly drops after layer 20.⁴

Our results indicate the LLMs can distinguish between words and nonwords. But this distinction can be attributed to the distributional properties of words: it might be the case that models are recognizing common sequences of tokens, rather than identifying whole words. To test this hypothesis, we repeat the same experiment, this time using the *penultimate* token representation, for words three tokens or longer. By definition, such words also as frequently co-occur with the initial sub-word tokens in their words as the final tokens. Our results (Fig. 2b, **orange line**) show that the probe only reaches 61% classification accuracy, indicating that the high classification accuracy does not stem from token co-occurrence, but is tied to the presence of a whole word. See App. A for an analysis of misclassified words and nonwords.

⁴We perform a similar experiment using a dataset of morphologically plausible nonwords (ARC Nonword Database; Rastle et al. 2002), which shows a similar trend. See App. A.

Overall, our results show that language models internally represent words and nonwords differently. This distinction is gradually developed in the model’s early layers and maintained throughout its middle layers. These findings support the hypothesis that the model performs a detokenization process and suggests where this process occurs. Building on these results, we next investigate how sub-word tokens are combined into word representations across model layers, and explore the internal mechanisms that facilitate this transformation.

4 EXTRACTING WORD IDENTITY FROM LLM HIDDEN STATES

We have so far observed that LLMs can differentiate between words and nonwords, suggesting an internal detokenization process specific to word composition. We next dig into this process, by asking whether we can directly extract word identity from the hidden states of sub-word tokens. We start by considering single-token words (Sec. 4.1), and then move on to multi-token words (Sec. 4.2).

4.1 SINGLE-TOKEN WORDS

We first consider in-vocabulary words, which are mapped to single tokens by the tokenizer. Naively, such words don’t tell us much about detokenization, as they are represented using only one token. To address this, we artificially split them into multiple sub-word tokens. For example, we take the single-token word “cats” and split it into two tokens: “ca” and “ts”. We hypothesize that if the model performs detokenization, it will represent the last token of the word (“ts”) similarly to the original word token (“cats”). We iterate the WIKITEXT-103 dataset (Merity et al., 2017) and randomly split each single-token word longer than three characters into 2–5 sub-words tokens. We then feed the model the new sequence of tokens preceded by the last 100 tokens that came before each split word in the original text as context.⁵

To measure the similarity between the representation of the final token and original word, we apply the logit lens method (nostalgebraist, 2020), an interpretability method that maps a hidden representation of a given token to the word whose vector in the output unembedding matrix is most closely aligned with the hidden representation. We note that this technique is typically used to inspect model predictions of the *next* token in intermediate layers. As we aim to identify how LLMs represent the *current* word, we use the *input* embeddings matrix rather than the unembedding one.⁶

We study four LLMs: Llama2-7B, Llama3-8B (Dubey et al., 2024), Mistral-7B (Jiang et al., 2023), and Yi-6B (AI et al., 2024). For each layer, we report the rate of retrieval—the proportion of words for which the closest vector in the vocabulary space is the original (single token) word.

Our results for the Llama2-7B are shown in Fig. 3a.⁷ Starting at layer 8, the hidden state of the final token is mapped to the original word with high accuracy, which peaks at more than 80% in layer 15. Interestingly, this accuracy then starts to decline, possibly because the model transitions into representing the prediction of the next token. We also find that 93.2% of all split words are correctly mapped to the original word in *at least one* model layer (see App. C.1).

Our results indicate that LLMs perform a detokenization process in cases where a single-token word is split into multiple tokens. This process assigns the final token with a hidden representation similar to that of the full, single-token word. But **how robust is this process?** To address this question, we consider a different way of splitting a word into sub-tokens—adding typos. For each word, we randomly flip two adjacent characters, delete a character, or insert a new character (see App. D for more details). Much like our previous experiment, this process splits the word into multiple tokens, as corrupted words are rarely found in BPE lexicons. We repeat the same experiment as above, aiming to study whether models map the last token to the original (correctly-spelled) word.

⁵We observe similar trends, in this and in other experiments in this section, when passing the split words *without* their context, albeit with slightly lower rates of retrieval.

⁶We also compute a second measure, cosine similarity, to compare between the hidden representations and input embeddings. We find similar results to those obtained using logit lens. For details, see App. F.

⁷Results for the other models (App. C) show a similar trend.

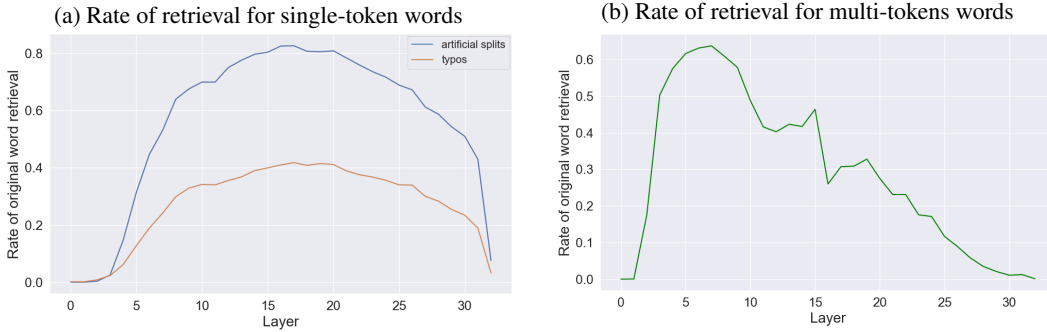


Figure 3: (3a) Logit lens rate of retrieval of single-token words artificially split (blue line) and split due to typos (orange). In both cases, we see an increasing rate of retrieval after the 4th layer, peaking in the middle (16–17 layers) and then dropping; (3b) Patchscopes rate of retrieval for multi-token words. In this case results peak in the 5th layer and then start to decline.

Our results for Llama2-7B (Fig. 3a) show a similar trend, though less pronounced: the model correctly matches about 40% of corrupted words with the original, single-token, uncorrupted word.⁸ We also note that the trend across layers persists, with accuracy peaking around layer 15.

4.2 MULTI-TOKEN WORDS

We have seen that LLMs are able to detokenize single-token words that are artificially broken into sub-words back into their correct (single-token) word. This indicates that these words are part of the model’s inner lexicon. But what about multi-token words? By definition, these words do not have a single embedding vector in the BPE vocabulary, so we cannot compare them to any existing vector, and thus cannot use logit lens.

Instead, we use the Patchscopes technique (Ghandeharioun et al., 2024), which interprets the hidden representation of a model using the model’s language abilities. In particular, we feed the model with the prompt “Repeat this word twice: 1) X 2)”, where X is the hidden representation of the last word token. Our hypothesis is that if the multi-token word is found in the model’s internal lexicon, then it will “understand” its representation in the input layer as well, and successfully repeat it by generating all tokens of the original word (see Fig. 1, right).

We evaluate the model by computing the rate of retrieval—the proportion of times it generates the correct (multi-token) word. We use the same models and dataset as in the previous experiments, though in this case we do not further split or add typos, as the words are already split into multiple tokens. Particularly, we feed each multi-token word (along with its 100-tokens context, as before) to the model, and extract the last token’s representation from each layer. We then use Patchscopes’ prompt in a new run to check if the model regenerates the original multi-token word.

Our Llama2-7B results (Fig. 3b) show a striking trend: when feeding the model with vectors from layers 5–7, it is able to repeat the word in 64% of the cases, despite never seeing this vector as input before.⁹ As to the different layers, we observe a similar trend to the single-token results: retrieval rates increase rapidly at some point (here earlier than in previous experiments, around layer 3–4), reach a peak at layer 7, and then start to decline. Combined, these results suggest that the model treats multi-tokens words as if they were in the vocabulary but split into sub-word tokens, indicating a latent vocabulary that expands beyond the tokenizer’s limitations. Interestingly, we observe that 22.6% of the multi-token words are never successfully decoded from any internal layer, hinting that they might not be represented in the model’s inner lexicon.

⁸Here, 66% of the words are correctly mapped in at least one layer.

⁹Overall, 77.4% of the words are repeated correctly in at least one layer.